Logged in

Select Mode

Preventive Mode

Logout

📊 DQDash - AI Data Quality Assistant

🛡 Preventive Mode - AI Database Design Advisor

Describe your business idea and the system will suggest an optimized database schema.

Enter Business Description

The system should support:
- instructors who create courses
- students who enroll in courses
- courses with multiple lessons
- student progress tracking
- course ratings and reviews
- payments for course purchases

Select LLM Model

phi3

Generate Database Design

📋 AI Generated Database Design

⚠ This AI-generated schema is a draft recommendation. Final validation by a database architect is recommended.

Clear Result

Database Design

SECTION 1: DATABASE DESIGN

Tables and Columns with Data Types:

- Instructors (InstructorID INT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), PasswordHash CHAR(64)) -- Primary Key is InstructorID. Foreign key for Courses table will be CourseCreator_FK.
- Students (StudentID INT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), Age INT) -- Primary Key is StudentID.
- Courses (CourseID INT, Title VARCHAR(255), Duration_Weeks SMALLINT, MaxStudents INT, InstructorID INT) -- Foreign key for Students table will be CourseTeacher_FK and to the instructors table by courseCreator_fk.
- Lessons (LessonID INT PRIMARY KEY AUTO_INCREMENT, Title VARCHAR(255), Description TEXT, Duration INTERVAL) -- Foreign keys for Courses lessons are CourseLessonsFK and InstructorID FK links back to instructors table.
- Enrollments (EnrollmentID INT PRIMARY KEY AUTO_INCREMENT, StudentID INT, CourseID INT, DateJoined DATE, CompletionStatus BOOLEAN) -- Foreign keys for Students is the enrolledStudent and for Courses by courseLesson FK which links back to Lessons.

- Ratings (RatingID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, CourseID INT, Rating TINYINT CHECK(Rating >= 1 AND Rating <= 5), Timestamp DATETIME) -- Foreign keys for Users is the ratedUser and to Courses by courseFK.
- Payments (PaymentID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, Amount DECIMAL(10,2), CourseID INT, DatePaid DATE, Status VARCHAR(50)) -- Foreign keys for Users is the paymentUser and to Courses by courseFK.
- Students (StudentID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), Age INT) - A junction table StudentCourses can be introduced here for many students enrolling into multiple courses if needed.
- Instructors (InstructorID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255)) - A junction table CourseTeachers can be introduced here for many instructors teaching multiple courses if needed.
- Users (UserID INT PRIMARY KEY AUTO_INCREMENT) -- This would include both students and payments entities, this will ensure that each user has a unique identifier across the platform to associate them with their respective records in Students or Payments tables using foreign keys studentFK/paymentUser FK.
- CourseReviews (ReviewID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, CourseID INT, Content TEXT, Rating SMALLINT CHECK(Rating >= 1 AND Rating <=5), Date DATE) -- Foreign keys for Users is the reviewUser and to Courses by courseFK.

2. Normalization Explanation:

- First normal form (1NF): Ensure all entries are atomic, no repeating groups or arrays in tables; implemented with unique student/user IDs.
- Second normal form (2NF): Create non-repeating group data and establish a primary key for each table that's not just auto-incremented without reference to another column. All foreign keys here have been established based on this principle, ensuring no partial dependency exists except in the junction tables where appropriate splitting is needed as StudentCourses/CourseTeachers if courses can be taught by multiple instructors and vice verssal for students taking several courses which requires additional normalization into a proper many-to-many relationship.
- Third normal form (3NF): Ensure that all fields are only dependent on the primary key, eliminating transitive dependency within tables; already achieved as CourseTeacher_FK and InstructorID in Courses ensure no non-key attributes depend on each other directly or indirectly through a third column.

SECTION 3: SQL CREATE TABLE SCRIPT

```sql
CREATE TABLE instructors (
    InstructorID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Email VARCHAR(255) UNIQUE,
    PasswordHash CHAR(64) -- For simplicity assuming hashed passwords for storage. Do NOT store plain text.
);

CREATE TABLE students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Email VARCHAR(255) UNIQUE,
    Age INT NOT NULL CHECK (Age > 0) -- Ensuring age is positive.
);

CREATE TABLE courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255),
    Duration_Weeks SMALLINT UNSIGNED,   -- No fractional weeks for simplicity of database design. CHECK to accept only positive integers or reasonable domain here if n
    MaxStudents INT NOT NULL CHECK(MaxStudents > 0), -- Enforcing non-negative course capacity and that courses will not enroll in negative numbers of students, which
    InstructorID INT UNIQUE,     // Uniqueness to ensure each instructor can only teach one Course at once (one-to-many relationship), assuming no course sharing or di
    FOREIGN KEY(InstructorID) REFERENCES instructors(InstructorID) ON DELETE SET NULL -- On delete set null to avoid foreign key constraint violation, assuming that ev
);

CREATE TABLE lessons (
    LessonID INT PRIMARY KEY AUTO_INCREMENT,
```

```
    Title VARCHAR(255),  -- Assuming each Course has exactly one unique identifier for its main content or could be linked to many-to-many relationship if courses have
    Description TEXT,
    Duration INTERVAL NOT NULL -- Storing interval directly might not always fit all RDBMS systems; this would typically use a DATETIME field for start and end time in
);

CREATE TABLE enrollments (
    EnrollmentID INT PRIMARY KEY AUTO0ROUNDED(8) -- Assuming MySQL RDBMS which uses DECIMAL type; might change based on the specific SQL dialect being used in producti
-- CHECK constraint for a complete course must be considered here if courses are required to fill all seats before enrollment possible as per business logic (e.g., no
    StudentID INT NOT NULL,  -- Foreign key here establishes many-to-many relationship between students and courses through Enrollments table which requires junction t
    CourseID INT NOT NULL,  -- Foreign key here establishes many-to-many relationship between students and courses through Enrollments table which requires junction ta
    DateJoined DATE NOT NULL,  -- Assumed to represent date when student enrolled into the course. May need change according based on business logic e.g., if a student
    CompletionStatus BOOLEAN NOT NULL, -- Optional field to track completion status of a course by the student. If required could also be tied back using more complex
    FOREIGN KEY(CourseID) REFERENCES courses(CourseID), -- This ensures integrity where only valid registered Courses can be joined by enrolled Students within respect
    FOREIGN KEY(StudentID) REFERENCES students(StudentID), -- Similar explanation for Student integrity as above, ensuring Students record reference back from Enrollme
```

SQL Script

```
SECTION 1: DATABASE DESIGN

Tables and Columns with Data Types:
- Instructors (InstructorID INT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), PasswordHash CHAR(64)) -- Primary Key is InstructorID. Foreign key
- Students (StudentID INT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), Age INT) -- Primary Key is StudentID.
- Courses (CourseID INT, Title VARCHAR(255), Duration_Weeks SMALLINT, MaxStudents INT, InstructorID INT) -- Foreign key for Students table will be CourseTeacher_FK and
- Lessons (LessonID INT PRIMARY KEY AUTO_INCREMENT, Title VARCHAR(255), Description TEXT, Duration INTERVAL) -- Foreign keys for Courses lessons are CourseLessonsFK an
- Enrollments (EnrollmentID INT PRIMARY KEY AUTO_INCREMENT, StudentID INT, CourseID INT, DateJoined DATE, CompletionStatus BOOLEAN) -- Foreign keys for Students is the
- Ratings (RatingID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, CourseID INT, Rating TINYINT CHECK(Rating >= 1 AND Rating <= 5), Timestamp DATETIME) -- Foreign keys fo
- Payments (PaymentID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, Amount DECIMAL(10,2), CourseID INT, DatePaid DATE, Status VARCHAR(50)) -- Foreign keys for Users is t
- Students (StudentID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), Age INT) - A junction table StudentCourses can
- Instructors (InstructorID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255)) - A junction table CourseTeachers can be
- Users (UserID INT PRIMARY KEY AUTO_INCREMENT) -- This would include both students and payments entities, this will ensure that each user has a unique identifier acro
- CourseReviews (ReviewID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, CourseID INT, Content TEXT, Rating SMALLINT CHECK(Rating >= 1 AND Rating <=5), Date DATE) -- Fore

2. Normalization Explanation:
- First normal form (1NF): Ensure all entries are atomic, no repeating groups or arrays in tables; implemented with unique student/user IDs.
- Second normal form (2NF): Create non-repeating group data and establish a primary key for each table that's not just auto-incremented without reference to another co
- Third normal form (3NF): Ensure that all fields are only dependent on the primary key, eliminating transitive dependency within tables; already achieved as CourseTea

SECTION 3: SQL CREATE TABLE SCRIPT
```sql
CREATE TABLE instructors (
    InstructorID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Email VARCHAR(255) UNIQUE,
    PasswordHash CHAR(64) -- For simplicity assuming hashed passwords for storage. Do NOT store plain text.
);

CREATE TABLE students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Email VARCHAR(255) UNIQUE,
    Age INT NOT NULL CHECK (Age > 0) -- Ensuring age is positive.
```
```

```
);

CREATE TABLE courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255),
    Duration_Weeks SMALLINT UNSIGNED,   -- No fractional weeks for simplicity of database design. CHECK to accept only positive integers or reasonable domain here if n
    MaxStudents INT NOT NULL CHECK(MaxStudents > 0), -- Enforcing non-negative course capacity and that courses will not enroll in negative numbers of students, which
    InstructorID INT UNIQUE,     // Uniqueness to ensure each instructor can only teach one Course at once (one-to-many relationship), assuming no course sharing or di
    FOREIGN KEY(InstructorID) REFERENCES instructors(InstructorID) ON DELETE SET NULL -- On delete set null to avoid foreign key constraint violation, assuming that ev
);

CREATE TABLE lessons (
    LessonID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255),  -- Assuming each Course has exactly one unique identifier for its main content or could be linked to many-to-many relationship if courses have
    Description TEXT,
    Duration INTERVAL NOT NULL -- Storing interval directly might not always fit all RDBMS systems; this would typically use a DATETIME field for start and end time in
);

CREATE TABLE enrollments (
    EnrollmentID INT PRIMARY KEY AUTO0ROUNDED(8) -- Assuming MySQL RDBMS which uses DECIMAL type; might change based on the specific SQL dialect being used in producti
-- CHECK constraint for a complete course must be considered here if courses are required to fill all seats before enrollment possible as per business logic (e.g., no
    StudentID INT NOT NULL,  -- Foreign key here establishes many-to-many relationship between students and courses through Enrollments table which requires junction t
    CourseID INT NOT NULL,  -- Foreign key here establishes many-to-many relationship between students and courses through Enrollments table which requires junction ta
    DateJoined DATE NOT NULL,  -- Assumed to represent date when student enrolled into the course. May need change according based on business logic e.g., if a student
    CompletionStatus BOOLEAN NOT NULL, -- Optional field to track completion status of a course by the student. If required could also be tied back using more complex
    FOREIGN KEY(CourseID) REFERENCES courses(CourseID), -- This ensures integrity where only valid registered Courses can be joined by enrolled Students within respect
    FOREIGN KEY(StudentID) REFERENCES students(StudentID), -- Similar explanation for Student integrity as above, ensuring Students record reference back from Enrollme
```

ER Diagram (Text)

SECTION 1: DATABASE DESIGN

Tables and Columns with Data Types:

- Instructors (InstructorID INT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), PasswordHash CHAR(64)) -- Primary Key is InstructorID. Foreign key for Courses table will be CourseCreator_FK.
- Students (StudentID INT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), Age INT) -- Primary Key is StudentID.
- Courses (CourseID INT, Title VARCHAR(255), Duration_Weeks SMALLINT, MaxStudents INT, InstructorID INT) -- Foreign key for Students table will be CourseTeacher_FK and to the instructors table by courseCreator_fk.
- Lessons (LessonID INT PRIMARY KEY AUTO_INCREMENT, Title VARCHAR(255), Description TEXT, Duration INTERVAL) -- Foreign keys for Courses lessons are CourseLessonsFK and InstructorID FK links back to instructors table.
- Enrollments (EnrollmentID INT PRIMARY KEY AUTO_INCREMENT, StudentID INT, CourseID INT, DateJoined DATE, CompletionStatus BOOLEAN) -- Foreign keys for Students is the enrolledStudent and for Courses by courseLesson FK which links back to Lessons.
- Ratings (RatingID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, CourseID INT, Rating TINYINT CHECK(Rating >= 1 AND Rating <= 5), Timestamp DATETIME) -- Foreign keys for Users is the ratedUser and to Courses by courseFK.
- Payments (PaymentID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, Amount DECIMAL(10,2), CourseID INT, DatePaid DATE, Status VARCHAR(50)) -- Foreign keys for Users is the paymentUser and to Courses by courseFK.
- Students (StudentID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255), Age INT) - A junction table StudentCourses can be introduced here for many students enrolling into multiple courses if needed.

- Instructors (InstructorID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(255), LastName VARCHAR(255), Email VARCHAR(255)) - A junction table CourseTeachers can be introduced here for many instructors teaching multiple courses if needed.
- Users (UserID INT PRIMARY KEY AUTO_INCREMENT) -- This would include both students and payments entities, this will ensure that each user has a unique identifier across the platform to associate them with their respective records in Students or Payments tables using foreign keys studentFK/paymentUser FK.
- CourseReviews (ReviewID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, CourseID INT, Content TEXT, Rating SMALLINT CHECK(Rating >= 1 AND Rating <=5), Date DATE) -- Foreign keys for Users is the reviewUser and to Courses by courseFK.

2. Normalization Explanation:

- First normal form (1NF): Ensure all entries are atomic, no repeating groups or arrays in tables; implemented with unique student/user IDs.
- Second normal form (2NF): Create non-repeating group data and establish a primary key for each table that's not just auto-incremented without reference to another column. All foreign keys here have been established based on this principle, ensuring no partial dependency exists except in the junction tables where appropriate splitting is needed as StudentCourses/CourseTeachers if courses can be taught by multiple instructors and vice verssal for students taking several courses which requires additional normalization into a proper many-to-many relationship.
- Third normal form (3NF): Ensure that all fields are only dependent on the primary key, eliminating transitive dependency within tables; already achieved as CourseTeacher_FK and InstructorID in Courses ensure no non-key attributes depend on each other directly or indirectly through a third column.

SECTION 3: SQL CREATE TABLE SCRIPT

```sql
CREATE TABLE instructors (
    InstructorID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Email VARCHAR(255) UNIQUE,
    PasswordHash CHAR(64) -- For simplicity assuming hashed passwords for storage. Do NOT store plain text.
);
```

```sql
CREATE TABLE students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(255),
    LastName VARCHAR(255),
    Email VARCHAR(255) UNIQUE,
    Age INT NOT NULL CHECK (Age > 0) -- Ensuring age is positive.
);

CREATE TABLE courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255),
    Duration_Weeks SMALLINT UNSIGNED,   -- No fractional weeks for simplicity of database design. CHECK to accept only positive integers or reasonable domain here if n
    MaxStudents INT NOT NULL CHECK(MaxStudents > 0), -- Enforcing non-negative course capacity and that courses will not enroll in negative numbers of students, which
    InstructorID INT UNIQUE,     // Uniqueness to ensure each instructor can only teach one Course at once (one-to-many relationship), assuming no course sharing or di
    FOREIGN KEY(InstructorID) REFERENCES instructors(InstructorID) ON DELETE SET NULL -- On delete set null to avoid foreign key constraint violation, assuming that ev
);

CREATE TABLE lessons (
    LessonID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255),  -- Assuming each Course has exactly one unique identifier for its main content or could be linked to many-to-many relationship if courses have
    Description TEXT,
    Duration INTERVAL NOT NULL -- Storing interval directly might not always fit all RDBMS systems; this would typically use a DATETIME field for start and end time in
);

CREATE TABLE enrollments (
    EnrollmentID INT PRIMARY KEY AUTO0ROUNDED(8) -- Assuming MySQL RDBMS which uses DECIMAL type; might change based on the specific SQL dialect being used in producti
-- CHECK constraint for a complete course must be considered here if courses are required to fill all seats before enrollment possible as per business logic (e.g., no
```

```
    StudentID INT NOT NULL,  -- Foreign key here establishes many-to-many relationship between students and courses through Enrollments table which requires junction t
    CourseID INT NOT NULL,  -- Foreign key here establishes many-to-many relationship between students and courses through Enrollments table which requires junction ta
    DateJoined DATE NOT NULL,  -- Assumed to represent date when student enrolled into the course. May need change according based on business logic e.g., if a student
    CompletionStatus BOOLEAN NOT NULL, -- Optional field to track completion status of a course by the student. If required could also be tied back using more complex
    FOREIGN KEY(CourseID) REFERENCES courses(CourseID), -- This ensures integrity where only valid registered Courses can be joined by enrolled Students within respect
    FOREIGN KEY(StudentID) REFERENCES students(StudentID), -- Similar explanation for Student integrity as above, ensuring Students record reference back from Enrollme
```

⬇ Download SQL Schema

⬇ Download ER Diagram Text

⬇ Download Design Report

⬇ Download Full Architecture Report